AWARE-US: Benchmark for Preference-Aware Resolution in Tool-Calling Agents

Mehmet Kurmaz

Abstract

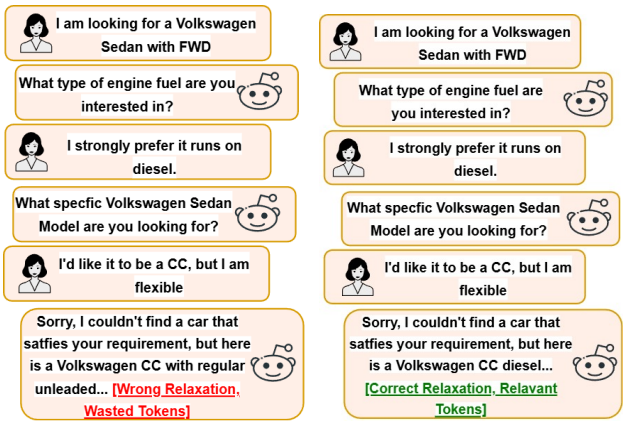
Tool-calling conversational agents querying structured databases often face two linked failures: underspecification (missing constraints needed to run a precise query) and infeasibility (the fully specified query returns an empty set because no item satisfies all constraints). Existing work in the literature respond with "no results" or relax constraints using ad hoc rules when faced an infeasible query, which can violate user intent by discarding requirements the user cares about most. We frame infeasibility handling as a preference-aware query repair problem: when a query is unsatisfiable, the agent should relax the least important/preferred constraints to the user. We propose three LLM-based methods for inferring relative constraint importance from dialogue: (1) local weighting, (2) global one-shot weighting, and (3) pairwise ranking. Results from extensive experiments show that the local-weighting approach, trained via supervised fine-tuning and direct preference optimization, achieves the best performance in car recommendation, aligning with user preferences in 48% of the time. Meanwhile, the global-weighting approach performs best on correct constraint relaxation, reaching 56%, where all 3 approaches perform stronger than prior infeasibility-resolution baselines.

We also introduce AWARE-US, a benchmark of 120+ persona-grounded queries requiring agents to (i) disambiguate a base request via conversation and (ii) resolve infeasibility in a way consistent with persona-implied preferences.

1. Introduction

Conversational agents are interactive systems designed to communicate with users in natural language to accomplish goals, ranging from open-ended social chat to goal-directed assistance (Wang et al., 2025; Acikgoz et al., 2024). A common high-level distinction is between open-domain dialogue and task-oriented dialogue (ToD), where the system helps the user complete a concrete task such as finding an item, booking a service, or answering questions grounded in an underlying knowledge source (Oruche et al., 2025; Chen et al., 2017; Wang et al., 2025).

In task-oriented settings (Wang et al., 2021; Hosseini-Asl et al., 2020), the agent's job is not only to produce fluent responses but to ground language in actions: it must interpret the user's intent, track constraints over turns (dialogue state tracking), decide what to do next (a dialogue policy), and finally execute a backend action such as a database lookup or API call before generating the response (Wang et al., 2025; Qin et al., 2023a). This view has been central to the ToD literature and motivates why backend connectivity is essential for real utility (Qin et al., 2023a).



(a) Undesired conversation. (b) Desired conversation.

Figure 1. Infeasible query handling. (a) Incorrect relaxation (wasted tokens) leads to an irrelevant recommendation, reduced user satisfaction, and poorer computational efficiency. (b) Correct relaxation (relevant result) yields a relevant recommendation, improving both user satisfaction and computational efficiency..

A known benchmark for multi-domain ToD is MultiWOZ, a large Wizard-of-Oz dataset with belief states and dialogue acts spanning multiple service domains, which helped standardize evaluation for state tracking and response generation in DB-backed assistants (Budzianowski et al., 2018).

However, tool-calling agents that interact with structured backends repeatedly run into two failure modes: underspecification, where the user's request lacks required constraints so the agent cannot form a precise query, and infeasibility, where the query is fully specified but no item satisfies all constraints (Zhang et al., 2024; Hao et al., 2025; Anonymous, 2025; Shim et al., 2025). These issues have been explored heavily in travel-planning settings. TravelPlanner provides a tool sandbox with 4M crawled records accessed via six tools, and includes 1,225 curated query–plan pairs(Xie et al., 2024)

Building on this, Ask-Before-Plan framework (Zhang et al., 2024) handles infeasibility using a static dependency graph

over "indefinite details," where clarification proceeds according to a topological priority order. Concretely, when an environmental response indicates an unfeasible detail, the agent asks for an alternative, but the choice of what to clarify next is governed by this pre-defined priority structure rather than inferred user-specific preferences. As a result, infeasibility resolution is guided by fixed domain assumptions.

By contrast, LLMs for Formal Verification (Hao et al., 2025) treats infeasibility as an SMT-solver signal: user intent is encoded as constraints, and an UNSAT result yields an unsat core with concrete failure reasons. The system then enters an interactive plan-repair loop where the LLM gathers tool evidence, proposes constraint changes, and solicits user agree/disagree feedback until the constraints become satisfiable though alignment relies heavily on repeated user input (Hao et al., 2025).

Meanwhile, Argus frames underspecification and infeasibility as tool-call disambiguation and evaluates on ClarifyBench (Anonymous, 2025), adding an explicit error-recovery step to repair the call or ask targeted clarifications often increasing user back-and-forth.

In light of these limitations, we formulate a new task: Preference-Aware Resolution. The goal is to handle infeasible queries, the empty-set problem in database systems (Mottin et al., 2013; 2014), without violating user intent. When an LM agent encounters an infeasible query, it should relax the constraint that is least important to the user, using preference cues expressed throughout the conversation, rather than relying on fixed domain priorities or repeated trial-and-error clarification. This moves tool-calling agents toward more negotiation-capable behavior, where repair decisions reflect what the user values most.

In addition, preference awareness should not stop at deciding which constraint to relax. Once a repaired query yields a non-empty candidate set, the agent must also use the inferred preference structure to select and rank results in a manner consistent with the user's priorities. Concretely, strong preferences expressed during the dialogue should be treated as optimization objectives over the remaining feasible set (e.g., minimizing or maximizing particular attributes), while weaker preferences may act as soft constraints that guide tie-breaking among otherwise acceptable options. Thus, Preference-Aware Resolution entails a coupled decision: (i) repairing infeasible queries by minimally sacrificing low-importance constraints, and (ii) producing recommendations that are Pareto-consistent with the user's expressed priorities within the feasible region, rather than arbitrary or default-ranked outputs.

To study this problem, we introduce AWARE-US, a benchmark of persona-grounded, multi-turn database queries in the car domain that requires agents to (i) disambiguate underspecified requests via conversation and (ii) resolve infeasibility in a way consistent with implied preferences. We then propose three LLM-based approaches for inferring constraint importance from dialogue: (i) local weighting, which estimates importance per constraint as it arises during clarification; (ii) global one-shot weighting, which assigns a normalized importance distribution over all extracted constraints after the dialogue; and (iii) pairwise ranking, which derives a global ordering by comparing constraints in pairs.

In summary, our contributions are:

- We introduce *Preference-Aware Resolution*, a new problem setting that studies how conversational LLM agents should handle infeasible (empty-set) queries by relaxing constraints in a way that preserves user intent.
- We propose three LLM-based frameworks for inferring relative constraint importance from dialogue and using it to guide query repair: local weighting, global one-shot weighting, and pairwise ranking.
- We construct **AWARE-US**, the first benchmark designed specifically for persona-grounded underspecification + infeasibility resolution in structured querying, requiring agents to both disambiguate requests via conversation and resolve infeasibility consistent with implied preferences.
- We develop a novel training strategy that combines supervised fine-tuning (SFT) (Ouyang et al., 2022) and direct preference optimization (DPO) (Rafailov et al., 2024) for this task, and conduct extensive evaluations and analyses across diverse settings to validate the effectiveness of our approaches.

1.1. Related Work

Language Model Based Agents Language agents have the capability to decompose complex tasks and arrive at solutions through a series of reasoned actions powered by Large language models (LLMs) (Xi et al., 2023; Deng et al., 2024; Wang et al., 2024) to conduct reasoning (Yao et al., 2022; Shinn et al., 2023), tool use (Qin et al., 2023b) , memory storage and retrival(Wen et al., 2024). A representative line is ReAct, which interleaves reasoning traces with environment actions to improve factuality and robustness in interactive settings (Yao et al., 2023).

Tool-use capability is also studied through training methods and benchmarks. Toolformer proposes self-supervised data generation to teach models when/how to call tools (Schick et al., 2023). Gorilla fine-tunes LMs to produce accurate API calls, often with retrieval to ground the call in documentation (Patil et al., 2023). Evaluation has consolidated around tool-use benchmarks such as API-Bank (tool-augmented dialogues with annotated API calls) and ToolBench (large-scale tool manipulation evaluation) (Qin et al., 2023b; Li et al., 2023b). However, most prior work assumes user instructions are clear enough for language agents to carry out, but in real settings instructions are often brief, which can introduce ambiguity and uncertainty.

Preference elicitation with agent Ambiguity is a core challenge for conversational agents because unclear requests

often trigger extra clarification turns, increasing interaction cost (Andukuri et al., 2024; Zhang et al., 2024; Anonymous, 2025). Prior work on clarification question generation studies how agents should ask targeted questions to resolve uncertainty efficiently (Li et al., 2023b; Zhang and Choi, 2023). In most agent querying settings, "ambiguous" is typically treated as underspecified (Zhang et al., 2024; Hao et al., 2025; Andukuri et al., 2024) missing constraints/information needed to form a precise action/query so existing systems emphasize information-seeking behaviors (slot filling and constraint elicitation). This view is also reflected in conversational semantic parsing benchmarks such as SParC and CoSQL (Yu et al., 2019a;b), which require context-dependent query construction. Preference elicitation is closely related, especially in conversational recommender systems, where dialogue refines user preferences and trade-offs (Jannach et al., 2021). By contrast, less work addresses fully specified but infeasible queries, where the agent must choose what to relax in line with user priorities rather than fixed heuristics or trial-and-error though database systems have studied this setting.

Infeasibility Resolution in Database Systems

In structured search, the empty-answer problem is commonly handled via query relaxation dropping/weakening conjunctive predicates until the query becomes satisfiable (Mottin et al., 2013; Ikeda et al., 2024; Mottin et al., 2014). The PVLDB probabilistic formulation models relaxation as a sequential optimization problem that trades off (i) the likelihood a user accepts a relaxation and (ii) the utility of resulting answers, but notes the search over relaxation sequences can be exponential in the number of predicates (Mottin et al., 2014). IQR operationalizes this with a query-relaxation tree and proposes one relaxation at a time, updating via user yes/no feedback often requiring multiple rounds when early relaxations are rejected (Mottin et al., 2014). More recent work improves result-set quality and efficiency, use MMR to return diversity-aware answers and highlight that enumerating relaxed queries is infeasible (Ikeda et al., 2024).

However, these database-centric approaches typically use pre-set objectives or acceptance models instead of inferring user-specific constraint importance from dialogue. As a result, handling infeasibility can can require exploring large parts of the exponential search space or interaction-heavy, motivating preference elicitation to guide repairs toward what the user actually values.

2. AWARE-US: Dataset Construction

AWARE-US is a benchmark for **preference-aware query disambiguation and infeasibility resolution** in structured retrieval settings. Each instance is a persona-grounded search task over a tabular database here, a car catalog from (CooperUnion, 2016), where an agent must (i) elicit missing constraints through dialogue and (ii) repair an infeasible query by relaxing constraints in a way that aligns with the user's latent preferences, rather than applying hand-coded heuristics. Pseudo algorithms provided in Appendix A and dataset examples in Appendix C.

2.1. Data source and query factorization

Let D denote a structured dataset with schema C (e.g., Make, Model, Year, MSRP, mpg, etc.). We partition constraints into (a) a **base slice** defined by a fixed set of "structural" attributes and (b) **additional constraints** that capture user-specific preferences. Concretely, each instance begins by sampling a base slice:

We define $U = \{\, x \in D \mid x[\text{Make}] = m,\ x[\text{Vehicle Style}] = s,\ x[\text{Transmission Type}] = t,\ x[\text{Driven_Wheels}] = w \,\}$, where Make, Vehicle Style, Transmission Type, and Driven_Wheels are the base columns. We restrict $|U|$ to be neither too small nor too large ($\texttt{BASE_MIN} \le |U| \le \texttt{BASE_MAX}$) to ensure each base slice admits meaningful variation while keeping search tractable.

The benchmark then constructs a set of additional constraints drawn from a mix of categorical predicates (e.g., Model, Vehicle Size, Engine Fuel Type) and numeric inequalities (e.g., Year $\ge y$, city mpg $\ge c$, MSRP $\le p$). The agent's goal is to infer these constraints from persona-driven dialogue, and then resolve infeasibility when the full set cannot be satisfied.

2.2. Minimal infeasibility (MUS-4) as the core hardness principle

To ensure that infeasibility is non-trivial (Reiter, 1987) and that "relaxing something" is necessary, we generate instances whose additional constraints form a **minimal unsatisfiable subset of size 4 (MUS-4)** (Liffiton and Sakallah, 2008; Bacchus and Katsirelos, 2015) within the base slice U.

This construction guarantees that the query is infeasible **only because of the interaction of all four constraints**, and that dropping any one constraint yields at least one feasible result. This directly targets our research question: *can an agent relax the right constraint(s) according to user preference, rather than relax arbitrarily?*

Operationally, we implement MUS-4 generation efficiently using bitset intersections over U. For each candidate predicate a, we precompute a bitset $b(a) \in \{0,1\}^{|U|}$ indicating which rows satisfy a. Then feasibility checks reduce to fast bitwise operations (e.g., $b(a_i) \wedge b(a_j) \wedge b(a_k)$). We enumerate "good triples" (a_i, a_j, a_k) with non-empty intersection and search for a fourth predicate a_ℓ such that the four-way intersection becomes empty, while each of the three-way intersections that include a_ℓ remains non-empty. This guided search increases yield compared to naive quadruple enumeration while preserving the MUS property.

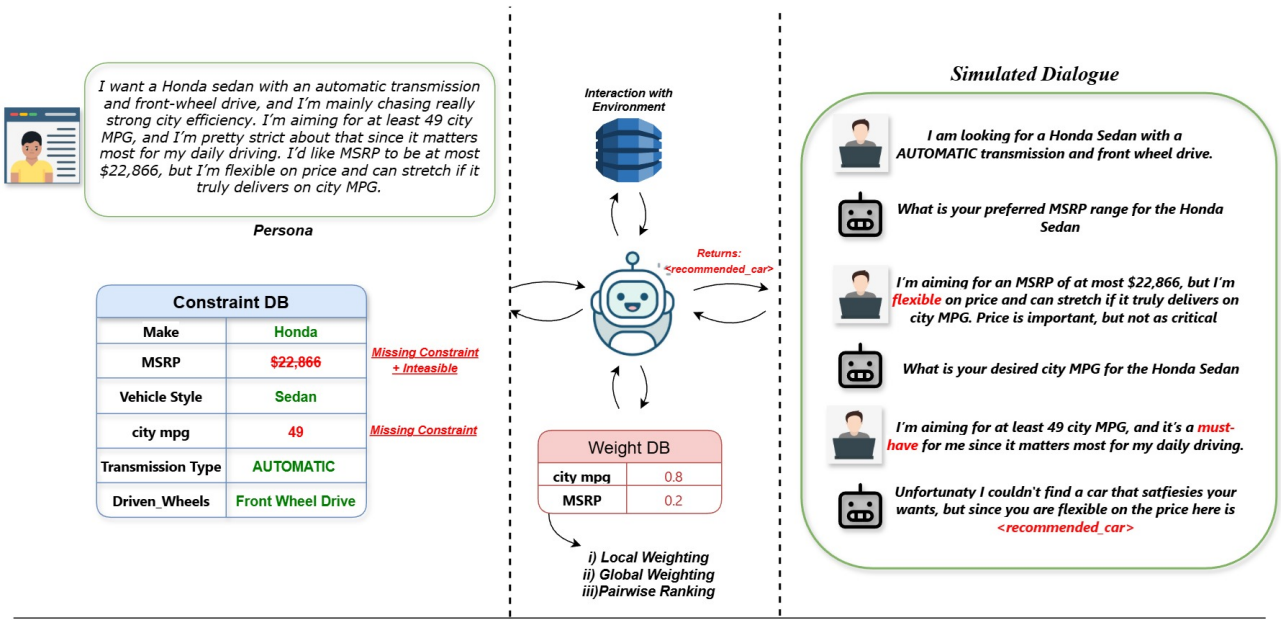


Figure 2. Overview of our Infeasibility-Aware Resolution Framework: Given a user query, specifying desired attributes and soft preferences, our framework elicits missing constraints via dialogue, extracts the resulting constraint set, and estimates preference weights (local weighting, global weighting, or pairwise ranking). The weighted constraints are used to query the constraint database; if the query is infeasible, the agent relaxes the lowest-importance constraint(s) to recover a feasible candidate and recommends optimized result based on user preferences.

2.3. Predicate generation with quality-preserving numeric thresholds

A major challenge is generating additional constraints that are neither vacuous nor absurdly extreme. For categorical columns, we include values with sufficient support in U to avoid degenerate predicates. For numeric columns, thresholds are derived from the empirical distribution within U, using a mixture of mid-to-high quantiles and near-extreme values. This is critical for constructing MUS-4 instances: near-extreme thresholds make infeasibility possible, while quantile-based thresholds keep constraints plausible and semantically meaningful.

To prevent low-quality instances that are technically MUS-4 but effectively "loose," we compute the sizes of the feasible sets obtained by dropping each constraint:

$$c_j \;=\; \left| U \cap \bigcap_{i \neq j} a_i \right|, \qquad j = 1, \ldots, 4,$$

and filter instances whose aggregate looseness (e.g., $\sum_j c_j$) exceeds a threshold. This enforces that relaxation produces a *small but non-empty* candidate set, which better matches real empty-result scenarios where small changes can recover viable options.

2.4. Oracle weights, relaxation target, and gold recommendation

To evaluate whether an agent's repairs align with user intent, each instance includes an oracle preference signal over the four constraints. We sample nonnegative weights and normalize:

$$\mathbf{w} \sim \mathrm{Uniform}(0.2, 1.0)^4, \qquad w_i \leftarrow \frac{w_i}{\sum_{j=1}^{4} w_j},$$

and treat the lowest-weight constraint as the intended relaxation:

$$a^\star \;=\; \arg\min_{a_i \in A} w_i.$$

Given A is MUS-4, relaxing $a^\star$ guarantees feasibility. The oracle then selects a recommended item from the feasible set $U \cap \bigcap_{a \in A \setminus \{a^\star\}} a$ using a soft scoring rule that rewards satisfying higher-weight constraints and uses normalized numeric desirability (e.g., newer year, higher mpg, lower MSRP) as applicable. This yields two gold signals: (i) the **chosen relaxation** and (ii) a **recommended record**.

This design isolates the central evaluation question: when faced with an empty result set, can the agent infer which constraint is least important *from persona-consistent dialogue* and relax that one?

2.5. Personas and dialogue supervision

Each query instance is paired with a **persona** describing the user's preferences in natural language. Personas for AWARE-US are generated using **GPT-5** (OpenAI, 2025) with prompts that encourage realistic, internally consistent preference descriptions (Jandaghi et al., 2024). The persona is treated as latent user state: it is visible to the Roleplayer (the simulated user) but not to the Questioner/Agent in interactive settings. Persona generation prompt and example personas provided in Appendix B.

Setting	# additional constraints	# instances
S1: MUS-4 (one unique repair)	4	40
S2: MUS-4 (any-one repair)	4	40
S3: MUS-2 (any-one repair)	2	40

Table 1. AWARE-US benchmark settings and sizes.

2.6. Benchmark settings

AWARE-US contains three controlled settings that vary how many additional constraints are present and how *sensitive* feasibility is to relaxing constraints. Let U denote the base slice induced by the base query, and let $A = \{a_1, \ldots, a_k\}$ denote the set of additional constraints.

Setting S1: MUS-4 (unique repair). This setting uses $k = 4$ additional constraints and is constructed to have a *unique single-constraint repair*:

$$U \cap \bigcap_{i=1}^{4} a_i \;=\; \varnothing, \qquad \exists!\, j \in \{1,2,3,4\} \text{ s.t. } U \cap \bigcap_{i \neq j} a_i \;\neq\; \varnothing.$$

Equivalently, relaxing exactly one specific constraint yields a non-empty result set, while relaxing any other single constraint does not.

Setting S2: MUS-4 (any-one repair). This setting also uses $k = 4$ additional constraints, but is constructed so that *any single constraint* can be relaxed to recover feasibility:

$$U \cap \bigcap_{i=1}^{4} a_i \;=\; \varnothing, \qquad \forall j \in \{1,2,3,4\},\; U \cap \bigcap_{i \neq j} a_i \;\neq\; \varnothing.$$

Thus, every drop-one variant is feasible, yielding four distinct one-constraint repairs.

Setting S3: MUS-2 (any-one repair). This setting uses $k = 2$ additional constraints and is constructed so that relaxing either constraint yields feasibility where, either single-constraint relaxation produces a non-empty result set.

3. Infeasibility Task

We study **preference-aware infeasibility resolution**, aiming problems we have listed previously. Let D be the database and let a user query induce a constraint set

$$C \;=\; C_{\text{base}} \cup C_{\text{add}},$$

where C_{base} defines a "base slice" (e.g., make/style/transmission/drivetrain) and C_{add} are additional constraints elicited via dialogue (e.g., Year $\geq y$, MSRP $\leq b$, fuel type, size, model). Feasibility is checked by

$$F(C) \;=\; \{x \in D \mid x \models C\}.$$

If $F(C) \neq \varnothing$, the agent recommends from $F(C)$. If $F(C) = \varnothing$, the agent must **repair** the query by relaxing some subset of C_{add} while preserving user preferences (Koudas et al., 2006). Task prompts are provided in Appendix D and example dialogue in Appendix F.

Baseline: solver-style satisfiability + repair. As a baseline, we reimplement the "LLM + verifier/solver" style used in (Hao et al., 2025): the LLM produces structured constraints, and a deterministic executor verifies satisfiability (here: database filtering) and enables systematic repairs when infeasible. This baseline reliably detects infeasibility, but it does not specify which constraint to relax in a preference-aligned way, for that reasons it is a good comparison to see what happens when agent ignores user preference when relaxing constraints.

Preference-aware repair: three variants. All methods share the same outer loop: (i) run clarification to instantiate C_{add}; (ii) test feasibility; (iii) if infeasible, relax constraints in a preference-guided order until feasible; and (iv) if multiple feasible items remain, rank candidates by a soft satisfaction score.

We explore three ways to infer the relative importance of constraints $c_i \in C_{\text{add}}$:

Local weighting. For each slot i, the agent asks one clarification question and receives a slot-specific answer. From that *local* exchange it extracts (i) a concrete constraint c_i (e.g., MSRP $\leq$ 20,000, Model = `"Civic"`) and (ii) an importance score $w_i \in [0, 1]$ inferred from the user's language in that same turn. If the full set is infeasible, the repair step relaxes constraints in increasing importance, i.e., relax the smallest w_i first until the feasible set becomes non-empty.

Global one-shot weighting. Here the agent still extracts the constraint set $C = \{c_i\}_{i=1}^{m}$ from the dialogue, but defers weighting until the end. After the full transcript S is collected, the agent runs a single holistic pass to assign weights jointly:

$$(w_1, \ldots, w_m) = f(S, C), \qquad w_i \geq 0,\; \sum_{i=1}^{m} w_i = 1.$$

Because f conditions on the entire conversation, it can incorporate cross-turn trade-offs more consistently than local weighting. Relaxation again proceeds by removing constraints in ascending w_i until feasibility is restored.

Pairwise ranking (Qin et al., 2024). Instead of predicting absolute weights, the model decides relative importance through pairwise comparisons. Given two constraints (c_i, c_j) and transcript S, it predicts which should be preserved:

$$c_i \succ c_j \quad \text{or} \quad c_j \succ c_i \quad \text{(or tie)}.$$

Aggregating these pairwise outcomes yields an ordering over constraints, and the relaxation sequence follows the least-preferred constraints first. This approach focuses directly on the *ordering* needed for repair, without requiring calibrated numeric weights.

In all three cases, once a feasible set $F(\cdot)$ is obtained after relaxing the least-important constraint(s), the agent chooses an item x by maximizing a weighted satisfaction score over remaining constraints. Our full implementation follows

this pipeline end-to-end (clarification → constraint extraction → feasibility check → preference-aware relaxation → recommendation).

3.1. Evaluation

We evaluate preference-aware infeasibility handling along three complementary dimensions that capture (i) whether the agent successfully completes the conversational elicitation stage, (ii) whether it produces a feasible plan and recommendation, and (iii) how well its repairs align with an oracle notion of the "correct" relaxation and choice. Concretely, our evaluation is organized into the following three metric categories: 1) Dialogue completion, 2) Planning outcome, 3) Oracle agreement.

For each model and method, we run the full end-to-end pipeline over all benchmark instances and report aggregated scores. For oracle-based metrics, we compute agreement only over the subset of instances where the comparison is well-defined. Full metric definitions and computation details are provided in Appendix E.

4. Supervised Finetuning and Direct Preference Optimization

We propose a novel training approach for this task to test the effectiveness of training our benchmark where we take a common approach of combining supervised finetuning (SFT) (Ouyang et al., 2022) and direct preference optimization (DPO)(Rafailov et al., 2024) where SFT to learn instruction-following, then DPO to refine behavior from preference pairs (Dettmers et al., 2023; Tunstall et al., 2023). We train a persona-conditioned Roleplayer based on Qwen/Qwen3-8B (Yang et al., 2025) with a two-stage pipeline that separates "format and coverage" from "preference-sensitive repair." We first deduplicate benchmark items by the pair (q, u), where q is the base query sentence and u is the persona text, then perform a seeded 80/20 item-level train/test (this split includes all benchmark settings). In stage one, we apply QLoRA-based (Dettmers et al., 2023) supervised fine-tuning (SFT) (Ouyang et al., 2022) on the training split to enforce two behaviors: (i) when asked about a slot, the model must surface any persona-backed constraint value rather than omitting it, and (ii) it must always output a single valid JSON object in a fixed schema. Each example is formatted as a chat prompt containing (q, u) plus a slot-specific clarification question; the target completion is the schema-conforming JSON for that slot. We optimize the standard maximum-likelihood objective on the completion tokens only, i.e.,

$$\max_{\theta} \sum_{(x,y)\in\mathcal{D}_{\text{SFT}}} \sum_{t\in\text{completion}(y)} \log \pi_\theta(y_t \mid x, y_{<t}),$$

where masking restricts the loss to the JSON completion to improve schema stability. Because some benchmark variants contain two additional constraints while others contain four, we generate positive supervision for all slots present in an item and include a small number of "absent slot" prompts to teach correct abstention while staying in-schema.

In stage two, we use Direct Preference Optimization (DPO) (Rafailov et al., 2024) to teach which constraint to relax under infeasibility. SFT alone does not reliably learn preference-faithful repairs when the full constraint set yields an empty result, so we construct preference pairs (x, y^+, y^-) from training items, where x encodes the query/persona context and infeasibility state, y^+ is the oracle-consistent repair outcome (correct relaxation and downstream resolution), and y^- is a plausible but incorrect alternative. DPO trains a policy π_θ relative to a fixed reference π_{ref} (initialized from the SFT model) by minimizing

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}_{\text{DPO}}}\left[\log \sigma\Big(\beta(\Delta_\theta(x) - \Delta_{\text{ref}}(x))\Big)\right],$$
$$\Delta_\theta(x) = \log \pi_\theta(y^+ \mid x) \;-\; \log \pi_\theta(y^- \mid x).$$

which encourages the preferred completion to be more likely than the rejected one, with strength controlled by β.

5. Experimental Results

For our experiments, we evaluated three large language models, Qwen3-8B (Yang et al., 2025), Gemma-2-9B-IT (Team et al., 2024), and Llama-3-8B-Instruct (Grattafiori et al., 2024), across the three benchmark settings listed in Table 1. These experiments were conducted using three different frameworks as well as a baseline method; further details are provided in Section 3.

Tables 2–4 report results for base large language models evaluated using our frameworks, with no additional training. Table 5 compares Qwen3-8B trained using our approach (see Section 4) against local and global baselines on our test set.

Dialogue-stage analysis. Across all settings, the dialogue stage is generally strong for the weighting-based agents: average extracted constraints are close to the gold count (e.g., ≈ 3.9 in MUS-4 and ≈ 1.8 in MUS-2), and slot completion is typically high (mid-to-high 90s for Qwen and Gemma). Pairwise ranking (PRP) shows consistently lower slot completion and fewer extracted constraints (e.g., 3.6 vs. 3.9 in MUS-4 for Qwen), which already hints that some of its downstream performance loss is attributable to weaker constraint recovery, not only ranking quality.

5.1. Base Model Comparison on Infeasibility Task

MUS-4 (one unique repair). Table 2 is the hardest setting: exactly one of four constraints must be relaxed. The formal-verification-style baseline achieves very high feasibility and recommendation rates (often near 100%), but its oracle agreement is poor (low relax match and especially low car match). This shows that "minimal change" repairs tend to prioritize what is easy to modify rather than what the user is willing to trade off. In contrast, local weighting yields the strongest preference-faithful outcomes in this strict regime, with the best car match for Qwen and Gemma.

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	**Avg Constraints**	**Slot Comp. (%)**	**SAT no relax (%)**	**SAT after relax (%)**	**UNSAT (%)**	**Reco rate (%)**	**Relax match (%)**	**Car match (%)**
Formal (Qwen3-8B)	3.9	97.6	4.9	95.1	0.0	100.0	14.6	4.9
Global (Qwen3-8B)	3.7	92.1	14.6	80.5	4.9	95.1	**65.9**	24.4
Weighted (Qwen3-8B)	3.9	96.3	9.8	56.1	34.1	65.9	46.3	**43.9**
PRP (Qwen3-8B)	3.6	89.0	31.7	56.1	12.2	87.8	48.8	17.1
Formal (Gemma-2-9b-it)	3.9	97.6	0.0	100.0	0.0	100.0	12.2	4.9
Global (Gemma-2-9b-it)	4.0	100.0	0.0	80.5	19.5	80.5	**63.4**	24.4
Weighted (Gemma-2-9b-it)	3.9	97.0	9.8	56.1	34.1	65.9	51.2	**46.3**
PRP (Gemma-2-9b-it)	3.8	93.9	12.2	80.5	7.3	92.7	61.0	24.4
Formal (Llama-3-8B-Instruct)	3.3	83.5	19.5	80.5	0.0	100.0	0.0	0.0
Global (Llama-3-8B-Instruct)	3.9	97.0	19.5	56.1	24.4	75.6	**31.7**	9.8
Weighted (Llama-3-8B-Instruct)	3.5	88.4	19.5	41.5	39.0	61.0	17.1	**14.6**
PRP (Llama-3-8B-Instruct)	3.3	82.9	41.5	51.2	7.3	92.7	14.6	9.8

Table 2. Summary results with grouped metrics for MUS-4 (one unique repair), refer Table 1. Percentages are reported as rates (%).

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	**Avg Constraints**	**Slot Comp. (%)**	**SAT no relax (%)**	**SAT after relax (%)**	**UNSAT (%)**	**Reco rate (%)**	**Relax match (%)**	**Car match (%)**
Formal (Qwen3-8B)	3.9	98.1	5.0	95.0	0.0	100.0	10.0	5.0
Global (Qwen3-8B)	3.9	96.2	12.5	87.5	0.0	100.0	40.0	25.0
Weighted (Qwen3-8B)	4.0	100.0	0.0	85.0	15.0	85.0	**55.0**	**47.5**
PRP (Qwen3-8B)	3.7	91.9	30.0	70.0	0.0	100.0	12.5	10.0
Formal (Gemma-2-9b-it)	3.9	98.1	15.0	85.0	0.0	100.0	0.0	0.0
Global (Gemma-2-9b-it)	4.0	100.0	5.0	95.0	0.0	100.0	**42.5**	**30.0**
Weighted (Gemma-2-9b-it)	4.0	99.4	5.0	82.5	12.5	87.5	25.0	15.0
PRP (Gemma-2-9b-it)	3.7	93.1	30.0	67.5	2.5	97.5	10.0	10.0
Formal (Llama-3-8B-Instruct)	3.7	92.5	25.0	75.0	0.0	100.0	0.0	0.0
Global (Llama-3-8B-Instruct)	3.9	97.5	17.5	72.5	10.0	90.0	**22.5**	**20.0**
Weighted (Llama-3-8B-Instruct)	3.8	94.4	27.5	45.0	27.5	72.5	20.0	7.5
PRP (Llama-3-8B-Instruct)	3.6	88.8	45.0	45.0	10.0	90.0	2.5	2.5

Table 3. Summary results with grouped metrics for MUS-4 (any unique repair), refer Table 1. Rates are reported as percentages.

	Dialogue Completion		Planning Outcome				Oracle Agreement	
Approach	**Avg Constraints**	**Slot Comp. (%)**	**SAT no relax (%)**	**SAT after relax (%)**	**UNSAT (%)**	**Reco rate (%)**	**Relax match (%)**	**Car match (%)**
Formal (Qwen3-8B)	1.7	86.6	22.0	78.0	0.0	100.0	22.0	17.1
Global (Qwen3-8B)	1.9	96.3	7.3	90.2	2.4	97.6	**65.9**	24.4
Weighted (Qwen3-8B)	1.8	87.8	22.0	75.6	2.4	97.6	63.4	**58.5**
PRP (Qwen3-8B)	1.6	79.3	36.6	63.4	0.0	100.0	48.8	22.0
Formal (Gemma-2-9b-it)	1.9	93.9	12.2	87.8	0.0	100.0	4.9	2.4
Global (Gemma-2-9b-it)	2.0	98.8	4.9	95.1	0.0	100.0	48.8	17.1
Weighted (Gemma-2-9b-it)	1.9	92.7	14.6	85.4	0.0	100.0	**56.1**	**53.7**
PRP (Gemma-2-9b-it)	1.8	89.0	24.4	75.6	0.0	100.0	41.5	22.0
Formal (Llama-3-8B-Instruct)	1.7	85.4	31.7	68.3	0.0	100.0	12.2	9.8
Global (Llama-3-8B-Instruct)	2.0	97.6	4.9	90.2	4.9	95.1	**53.7**	24.4
Weighted) (Llama-3-8B-Instruct)	1.8	87.8	26.8	61.0	12.2	87.8	41.5	**34.1**
PRP (Llama-3-8B-Instruct)	1.6	80.5	41.5	58.5	0.0	100.0	19.5	7.3

Table 4. Summary results with grouped metrics for MUS-2 (one unique repair), refer Table 1. Rates are reported as percentages.

Global one-shot weighting can improve relax match, but it is less consistent at translating that into the oracle's final recommendation, suggesting that transcript-level weights can still miss localized trade-off signals.

MUS-4 (any repair). Table 3 is more permissive: multiple relaxations can restore feasibility. As expected, most methods improve on feasibility. However, weighting remains advantageous for alignment: local weighting typically produces the highest car match (e.g., Qwen), indicating that even when feasibility is easy, preference signals help select the "right" repair and recommendation rather than an arbitrary satisfiable one. PRP remains weaker overall, which is consistent with its slightly lower slot completion and the added difficulty of turning pairwise comparisons into a stable repair-and-selection policy.

MUS-2 (one unique repair). Table 4 reduces complexity to two constraints, so performance depends more directly on modeling "which matters more." Here, local weighting shows the clearest gains across all model families, achieving the highest car match for Qwen, Gemma, and Llama. This supports the interpretation that local weighting is capturing persona-conditioned importance rather than benefiting from search heuristics.

5.2. Trained Approach

We trained Qwen3-8B (Yang et al., 2025) with outlined training approach in Section 3, since it was unclear which framework between local and global approaches performed better, we tested them again against trained and base models on our test split (includes all benchmark settings). .

Local pipeline. The first thing to note is that the trained model from Table 5 improves the dialogue-to-structure step,

which is the prerequisite for any preference-faithful repair. Under the LOCAL pipeline, the trained model reaches 100% slot completion versus 93.9% for the base model, and it also recovers slightly more constraints on average (3.28 vs. 3.08 parsed). This translates directly into stronger downstream behavior: the trained LOCAL system increases SAT-after-relaxation from 68% → 88%, reduces UNSAT from 20% → 12%, and improves the recommendation rate from 80% → 88%. Importantly, the gains are not just feasibility: preference alignment also improves, with relax match rising (44% → 52%) and car match (gated) rising substantially (36% → 48%). The gating is key: it indicates that when the model relaxes the same constraint as the oracle, it is also more likely to land on the oracle's final recommendation, evidence that training is helping the model carry persona information through to the end of the pipeline.

Global pipeline. For the GLOBAL pipeline from Table 5, the trained model does not improve slot completion (it remains 90.2% for both), and it actually parses slightly fewer constraints on average (2.96). Even so, it still produces meaningful gains in planning outcomes: SAT-after-relaxation improves (72% → 76%), while SAT-no-relaxation improves (20% → 24%) and overall UNSAT stays low at 4% for both. The more important story here is alignment: the trained GLOBAL system improves relax match (40% → 56%), but that improvement does not carry through to the final recommendation, since car match is unchanged at 36% (though it improves against base GLOBAL: 28% → 36%). This suggests that a holistic weighting pass can better identify which constraint should be relaxed, yet still struggles to consistently select the same item as the oracle, likely because the recommendation step depends on fine-grained trade-offs that are easier to preserve when weights are grounded in the slot-level exchanges (as in LOCAL).

6. Discussion

Overall, performance is dominated by two bottlenecks, (i) recovering the full set of persona-backed constraints through dialogue, and (ii) choosing the oracle-consistent relaxation when the query is infeasible. Methods that succeed at the first stage boost recommendation rate by avoiding "silent underspecification" and reducing downstream ambiguity. This helps explain why the trained variants (Table 5) generally improve SAT-after-relaxation and reduce UNSAT: the pipeline gets cleaner, better-structured inputs.

Across models and settings, weighted relaxation is the most reliable way to turn infeasible queries into feasible recommendations while maintaining preference alignment. The key evidence is the paired improvement in Relax match and Car match (gated): relaxing the same constraint as the oracle also increases the chance of selecting the same final item, suggesting the method isn't just "making something feasible," but doing so in a way that preserves the oracle's preference ordering. By contrast, the formal-verification-style baseline excels on feasibility (often near-perfect recommendation rate), but can lag on oracle agreement because "minimal change" repairs aren't inherently preference-aware small edits aren't necessarily the least important edits.

	Local		Global	
Metric	**Tr.**	**Base.**	**Tr.**	**Base.**
N	25	25	25	25
Avg slots	3.28	3.28	3.28	3.28
Avg parsed	3.28	3.08	2.96	2.96
Slot comp.	100.0%	83.9%	90.2%	90.2%
SAT (no relax)	0.0%	12.0%	24.0%	20.0%
SAT (after relax)	88.0%	68.0%	76.0%	72.0%
UNSAT	12.0%	20.0%	4.0%	4.0%
Reco rate	88.0%	80.0%	96.0%	96.0%
Oracle comp.	25	25	25	25
Relax match	52.0%	44.0%	56.0%	40.0%
Car match (gated)	48.0%	36.0%	36.0%	28.0%

Table 5. Comparison of Trained and Base Qwen3-8B Model under Local and Global Settings against test set.

Local vs. global weighting reveals a trade-off. Global weighting can be more consistent at identifying the least important constraint when preferences are expressed indirectly across turns, but it's more sensitive to imperfect parsing and can miss fine-grained intent needed for final selection. Local weighting, while simpler, benefits more from training because it ties weights to slot-level evidence; once SFT reduces missing constraints and stabilizes extraction, local weighting yields stronger end-to-end oracle agreement.

Finally, the MUS-4 settings show infeasibility resolution isn't one regime. When any relaxation yields feasibility, most methods look good on outcome, so the differentiator is oracle agreement. When only one relaxation is correct, the task becomes sharply preference-sensitive, widening the gap between preference-aware weighting and heuristic/minimal-change methods. This supports the paper's central claim: robust infeasibility resolution requires modeling which constraints matter to the user, not just finding any nearby satisfiable query.

7. Conclusion.

This paper introduced *Preference-Aware Resolution* for tool-calling agents: when a fully specified database query becomes infeasible, the agent should relax the *least important* constraint based on preference cues gathered through dialogue, rather than relying on fixed heuristics or repeated trial-and-error. To study this, we proposed three LLM-based importance-inference methods (see Section 3) and released AWARE-US, a persona-grounded benchmark that couples underspecification handling with preference-faithful infeasibility repair and recommendation selection. Across models and settings, the results consistently show that weighting-based strategies especially local weighting produce more intent-consistent repairs and higher agreement with oracle preferences than solver-style minimal-change baselines, supporting the claim that robust infeasibility handling benefits from explicitly modeling user-specific

constraint importance.

Further work. A direct next step is data augmentation of our benchmark, expanding AWARE-US in both size and diversity including additional domains, richer constraint formulations, more challenging infeasibility patterns within persona to better stress-test preference-aware repair under realistic conditions. On the optimization side, there is substantial room to extend our use of DPO: future work can explore stronger preference-pair generation (including hard negatives that mimic plausible but incorrect relaxations), trajectory-level (Zhang et al., 2024) preference optimization that evaluates the *entire* repair-and-recommend sequence rather than a single step, and hybrid training recipes that combine supervised learning for robust preference extraction with DPO-style objectives that better preserve fine-grained trade-offs during final selection.

8. Limitations.

Our experiments were constrained by computational limits, which restricted the scale of training, the breadth of hyperparameter sweeps (e.g., DPO parameters and pair construction strategies), and the number and size of models we could evaluate comprehensively across all settings. In addition, the current benchmark scope is limited: AWARE-US focuses on a structured car domain with a finite set of constraint types and instances, and it relies on persona-conditioned supervision signals (e.g., oracle weights/repairs) that may not fully capture the diversity, noise, and preference drift present in real user interactions or in other domains specially in TravelPlanning.

References

Emre Can Acikgoz, Dilek Hakkani-Tur, and Gokhan Tur. The rise of conversational ai agents with large language models. *emrecanacikgoz.github.io*, November 2024. URL https://emrecanacikgoz.github.io/Conversational-Agents/.

Chinmaya Andukuri, Jan-Philipp Fränken, Tobias Gerstenberg, and Noah D. Goodman. Star-gate: Teaching language models to ask clarifying questions, 2024. URL https://arxiv.org/abs/2403.19154.

Anonymous. Argus: Disambiguating user queries for tool-calling agents via uncertainty quantification. In *Submitted to ACL Rolling Review - May 2025*, 2025. URL https://openreview.net/forum?id=U1SpjHP8gh. under review.

Fahiem Bacchus and George Katsirelos. Using minimal correction sets to more efficiently compute minimal unsatisfiable sets. In *International Conference on Computer Aided Verification*, pages 70–86. Springer, 2015.

Paweł Budzianowski, Tsung-Hsien Wen, Bo-Hsiang Tseng, Iñigo Casanueva, Stefan Ultes, Osman Ramadan, and Milica Gašić. MultiWOZ - a large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5016–5026, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1547. URL https://aclanthology.org/D18-1547/.

Hongshen Chen, Xiaorui Liu, Dawei Yin, and Jiliang Tang. A survey on dialogue systems: Recent advances and new frontiers. *Acm Sigkdd Explorations Newsletter*, 19(2): 25–35, 2017.

CooperUnion. Car features and msrp. Kaggle dataset, December 2016. URL https://www.kaggle.com/datasets/CooperUnion/cardataset. Published 21 Dec 2016. Accessed 4 Jan 2026.

Yang Deng, An Zhang, Yankai Lin, Xu Chen, Ji-Rong Wen, and Tat-Seng Chua. Large language model powered agents in the web. In *Companion Proceedings of the ACM Web Conference 2024*, pages 1242–1245, 2024.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL https://arxiv.org/abs/2305.14314.

Aaron Grattafiori, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Yilun Hao, Yongchao Chen, Yang Zhang, and Chuchu Fan. Large language models can solve real-world planning rigorously with formal verification tools, 2025. URL https://arxiv.org/abs/2404.11891.

Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33:20179–20191, 2020.

Yuto Ikeda, Chuan Xiao, and Makoto Onizuka. An efficient diversity-aware method for the empty-answer problem. In *DOLAP*, pages 68–72, 2024.

Pegah Jandaghi, XiangHai Sheng, Xinyi Bai, Jay Pujara, and Hakim Sidahmed. Faithful persona-based conversational dataset generation with large language models. In *Proceedings of the 6th Workshop on NLP for Conversational AI (NLP4ConvAI 2024)*, pages 114–139, 2024.

Dietmar Jannach, Ahtsham Manzoor, Wanling Cai, and Li Chen. A survey on conversational recommender systems. *ACM Computing Surveys*, 54(5):1–36, May 2021. ISSN 1557-7341. doi: 10.1145/3453154. URL http://dx.doi.org/10.1145/3453154.

Nick Koudas, Chen Li, Anthony KH Tung, and Rares Vernica. Relaxing join and selection queries. In *Proceedings of the 32nd international conference on Very large data bases*, pages 199–210, 2006.

Belinda Z. Li, Alex Tamkin, Noah Goodman, and Jacob Andreas. Eliciting human preferences with language models, 2023a. URL https://arxiv.org/abs/2310.11589.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms, 2023b. URL https://arxiv.org/abs/2304.08244.

Mark H Liffiton and Karem A Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008.

Davide Mottin, Alice Marascu, Senjuti Basu Roy, Gautam Das, Themis Palpanas, and Yannis Velegrakis. A probabilistic optimization framework for the empty-answer problem. *Proceedings of the VLDB Endowment*, 6(14): 1762–1773, 2013.

Davide Mottin, Alice Marascu, Senjuti Basu Roy, Gautam Das, Themis Palpanas, and Yannis Velegrakis. Iqr: an interactive query relaxation system for the empty-answer problem. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of Data*, pages 1095–1098, 2014.

OpenAI. Gpt-5 documentation. https://openai.com/index/introducing-gpt-5/, 2025. Accessed: 2026-01-04.

Roland Oruche, Sai Keerthana Goruganthu, Rithika Akula, Xiyao Cheng, Ashraful Md Goni, Bruce W Shibo, Kerk Kee, Marcos Zampieri, and Prasad Calyam. A survey on the recent advancements in human-centered dialog systems. *ACM Computing Surveys*, 57(10):1–36, 2025.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL https://arxiv.org/abs/2203.02155.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. URL https://arxiv.org/abs/2305.15334.

Libo Qin, Wenbo Pan, Qiguang Chen, Lizi Liao, Zhou Yu, Yue Zhang, Wanxiang Che, and Min Li. End-to-end task-oriented dialogue: A survey of tasks, methods, and future directions, 2023a. URL https://arxiv.org/abs/2311.09008.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023b. URL https://arxiv.org/abs/2307.16789.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. Large language models are effective text rankers with pairwise ranking prompting, 2024. URL https://arxiv.org/abs/2306.17563.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL https://arxiv.org/abs/2305.18290.

Raymond Reiter. A theory of diagnosis from first principles. *Artificial intelligence*, 32(1):57–95, 1987.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Jeonghoon Shim, Gyuhyeon Seo, Cheongsu Lim, and Yohan Jo. Tooldial: Multi-turn dialogue generation method for tool-augmented language models, 2025. URL https://arxiv.org/abs/2503.00564.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, and Alexandre Ramé. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Kashif Rasul, Younes Belkada, Shengyi Huang, Leandro Von Werra, Clémentine Fourrier, Nathan Habib, et al. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*, 2023.

Hongru Wang, Lingzhi Wang, Yiming Du, Liang Chen, Jingyan Zhou, Yufei Wang, and Kam-Fai Wong. A survey of the evolution of language model-based dialogue systems: Data, task and models, 2025. URL https://arxiv.org/abs/2311.16789.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), March 2024. ISSN 2095-2236. doi: 10.1007/s11704-024-40231-1. URL http://dx.doi.org/10.1007/s11704-024-40231-1.

W Wang, Z Zhang, J Guo, Y Dai, B Chen, and W Luo. Task-oriented dialogue system as natural language generation. arxiv 2021. *arXiv preprint arXiv:2108.13679*, 2021.

Licheng Wen, Daocheng Fu, Xin Li, Xinyu Cai, Tao Ma, Pinlong Cai, Min Dou, Botian Shi, Liang He, and Yu Qiao. Dilu: A knowledge-driven approach to autonomous driving with large language models, 2024. URL https://arxiv.org/abs/2309.16292.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The rise and potential of large language model based agents: A survey, 2023. URL https://arxiv.org/abs/2309.07864.

Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents, 2024. URL https://arxiv.org/abs/2402.01622.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, and Chengen Huang. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

Tao Yu, Rui Zhang, He Yang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter S Lasecki, and Dragomir Radev. Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases, 2019a. URL https://arxiv.org/abs/1909.05378.

Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. Sparc: Cross-domain semantic parsing in context, 2019b. URL https://arxiv.org/abs/1906.02285.

Michael J. Q. Zhang and Eunsol Choi. Clarify when necessary: Resolving ambiguity through interaction with lms, 2023. URL https://arxiv.org/abs/2311.09469.

Xuan Zhang, Yang Deng, Zifeng Ren, See-Kiong Ng, and Tat-Seng Chua. Ask-before-plan: Proactive language agents for real-world planning, 2024. URL https://arxiv.org/abs/2406.12639.

A. Dataset Construction Algorithms

To see pseudocode algorithm for MUS-2/4 (any repair), refer to Algorithm 1 and for MUS-4(unique repair) refer to Algorithm 2

B. Persona Generation Prompts and Example Personas

```
You are a persona generator.

INPUT:
You will receive a JSON value that is either:
- an array of objects, or
- a single object.

Each object contains (at minimum) fields like:
- base_query_sentence (string)
- additional_constraints (array of constraint objects)
- constraint_weights (array of constraint objects with "weight" and possibly "is_unique_repair")
- unique_repair_constraint (optional)
- relaxation_policy (optional)
- chosen_relaxation (optional)

GOAL:
For each input object, generate ONE natural-language "persona" paragraph in first person that explains:
1) what the user wants (from base_query_sentence),
2) which constraints matter most vs least (from constraint_weights), and
3) how willing they are to compromise on each constraint (based on weights and/or chosen_relaxation).

OUTPUT RULES (VERY IMPORTANT):
- Output MUST be valid JSON only.
- Output MUST be:
  - a JSON array of strings (one persona per input object, same order) if input is an array, OR
  - a single JSON string if input is a single object.
- Do NOT output any extra keys, metadata, explanations, markdown, or commentary.

STYLE RULES:
- Write in a friendly, conversational tone.
- First person ("I'm looking for...", "I want...").
- 3-5 sentences per persona.
- Include mild, realistic motivation in the first sentence (based on vehicle style when possible):
  - Convertible/Coupe: "fun", "driver-focused", "weekend car"
  - Pickup: "workhorse", "capable", "everyday truck"
  - SUV: "practical", "confident", "bad weather"
  - Sedan/Hatchback: "comfortable", "easy to live with", "daily car"
  - Van: "haul people", "space", "no-drama"
  - Otherwise: generic "practical / a great fit"
- Avoid sounding robotic; vary phrasing slightly, but keep the same overall structure.

WEIGHT → FLEXIBILITY MAPPING:
Use constraint_weights sorted by descending weight to decide emphasis.
Convert each constraint into a natural phrase:
- Year >= Y  → "it being Y or newer"
- MSRP <= X → "keeping MSRP at most $X" / "staying under $X"
- city mpg >= N → "getting at least N city MPG"
- highway MPG >= N → "getting at least N highway MPG"
- Model == M → "it being a/an M" / "I really want a M"
- Vehicle Size == S → "it being S size"
- Engine Fuel Type == F → "it running on F"
(Keep original casing for model names; for fuel type, keep the exact value.)

Then describe priorities using these bands (choose phrasing that fits naturally):
- Top constraint (highest weight): "The most important thing for me is
... and I'm not very willing to compromise on that."
- High (next 1-2 constraints, weights roughly >= 0.30): "I also strongly
prefer ... / I care a lot about ... and I'm fairly set on that."
- Medium (~= 0.12-0.29): "I'm aiming for ... and I care about that, but I can
compromise a little if the rest is right."
- Low (<= 0.11): "I'd like ..., but I'm very flexible on that and can easily compromise."
```

```
SPECIAL RULES:
- If chosen_relaxation exists, that constraint MUST be described as the
softest/most flexible one (even if its weight isn't the smallest). Put it
in the final sentence if possible.
- If a constraint has is_unique_repair:true, treat it as LOW priority
unless it is also the top-weighted constraint (rare). If it's also
chosen_relaxation, definitely make it the softest.
- Mention all constraints in constraint_weights exactly once (don't drop any).
- Do not mention numeric weights or the words "weight", "unique repair", or "relaxation policy".

Now generate the personas for the following JSON input (remember: output JSON ONLY, personas ONLY):
{{INPUT_JSON_HERE}}
```

B.1. Example Personas, MUS-4(unique repair), MUS-4(any repair), MUS-2(any repair) accordingly.

I'm looking for a Lexus IS 250 sedan with an automatic transmission and rear-wheel drive, because I like that sporty-but-composed feel. The model is my top priority—I really want an IS 250 and I'm not very willing to compromise on that. I also strongly prefer it to be Midsize, since that size feels like the perfect balance for me. I'm aiming for at least 21 city MPG, and I care about that, but I can be a bit flexible if the car is otherwise a great match. For fuel, I'd like regular unleaded, but I'm extremely flexible there and can easily compromise on it.

I want a Mercedes-Benz sedan with an automatic transmission and rear-wheel drive, because I'm after that classic smooth luxury feel. The biggest must-haves for me are that it takes premium unleaded (required) and that it's specifically an E-Class, since those are core to what I'm looking for and I'm not really willing to compromise on them. I'd strongly prefer it to be 2016 or newer, but I could bend a little if everything else is perfect. I'm aiming for at least 23 city MPG, though I'm flexible on that and can compromise if the driving experience and model match are right.

I'm looking for a Buick sedan with an automatic transmission and front-wheel drive, and I'm really set on it being a Verano. The model matters most to me, so I'm not very willing to compromise on Verano specifically. I'd like it to run on premium unleaded (required), but I'm more flexible there and can compromise on fuel type if the Verano match is right.

C. Example Cell from Dataset (MUS-4 (any repair))


```
[
  {
    "base_query_sentence": "I am looking for a Chevrolet Extended Cab
    Pickup with a AUTOMATIC transmission and rear wheel drive.",
    "full_query_sentence": "I am looking for a Chevrolet Extended Cab
    Pickup with a AUTOMATIC transmission and rear wheel drive, gets at
    least 20 highway MPG, is large size, has MSRP at most $24,515, runs on
    flex-fuel (unleaded/E85).",
    "additional_constraints": [
      {
        "column": "highway MPG",
        "op": ">=",
        "value": 20
      },
      {
        "column": "Vehicle Size",
        "op": "==",
        "value": "Large"
      },
      {
        "column": "MSRP",
        "op": "<=",
        "value": 24515
      },
      {
        "column": "Engine Fuel Type",
        "op": "==",
        "value": "flex-fuel (unleaded/E85)"
      }
    ],
    "constraint_weights": [
      {
        "column": "highway MPG",
        "op": ">=",
```

```
        "value": 20,
        "weight": 0.16765467783968296
      },
      {
        "column": "Vehicle Size",
        "op": "==",
        "value": "Large",
        "weight": 0.3318273274571206
      },
      {
        "column": "MSRP",
        "op": "<=",
        "value": 24515,
        "weight": 0.3770969869897481
      },
      {
        "column": "Engine Fuel Type",
        "op": "==",
        "value": "flex-fuel (unleaded/E85)",
        "weight": 0.12342100771344829
      }
    ],
    "relaxation_policy": "relax_lowest_weight_constraint",
    "chosen_relaxation": {
      "column": "Engine Fuel Type",
      "op": "==",
      "value": "flex-fuel (unleaded/E85)",
      "weight": 0.12342100771344829
    },
    "recommended_car": {
      "Make": "Chevrolet",
      "Model": "Silverado 1500 Classic",
      "Year": 2007,
      "Engine Fuel Type": "regular unleaded",
      "Transmission Type": "AUTOMATIC",
      "Driven_Wheels": "rear wheel drive",
      "Vehicle Size": "Large",
      "Vehicle Style": "Extended Cab Pickup",
      "city mpg": 14,
      "highway MPG": 20,
      "MSRP": 21465
    },
    "meta": {
      "U_size": 43,
      "tight_score": 17,
      "drop_counts": [
        1,
        2,
        12,
        2
      ]
    },
    "persona": "I want a Chevrolet Extended Cab Pickup with an automatic
    transmission and rear-wheel drive\u2014something straightforward and
    easy to live with. I\u2019m pretty firm on keeping MSRP at no more
    than $24,515, because budget discipline matters most to me. I strongly
    prefer it to be a Large truck, though I could compromise a little on
    size if the overall deal is right. I\u2019m aiming for at least 20
    highway MPG, but that\u2019s more of a \u201cnice floor\u201d than a
    hard line. I\u2019d like it to run on flex-fuel (unleaded/E85), but
    I\u2019m flexible there if everything else fits."
  }
]
```


D. Infeasibility Task Prompts

D.1. ROLEPLAYER_SYS (System Prompt)

```
You are ROLEPLAYER_USER. You ONLY know the provided persona and the base
```

```
query.
Answer the assistant's question using ONLY information that is stated or
clearly implied in the persona.
IMPORTANT: When the assistant is clarifying a constraint, you should be helpful and specific:

if the persona provides ANY relevant value (e.g., a specific model name, a
year minimum, a budget cap),
you MUST state that value explicitly in your answer.
Do NOT say 'no preference' or omit the value if the persona mentions it.
Only say you have no preference if the persona truly contains no
information about that attribute.
If the assistant's question is a bit vague, still provide the most
relevant constraint value(s) from the persona.
Do not invent details that are not in the persona.
```

D.2. AGENT_ASK_SYS (System Prompt)

```
You are CLARIFYING_AGENT. Ask ONE short question to learn the user's
requirement for the given slot/field.
Do not propose relaxations. Do not ask multiple questions at once.
```

D.3. AGENT_EXTRACT_SYS (System Prompt)

```
You are a strict JSON generator. Extract (1) the constraint value from the
user's answer for the given slot,
and (2) an importance weight in [0,1] where 1=very important (must-have)
and 0=very flexible (easy to relax).
Output ONLY JSON.
```

D.4. agent_question: User Prompt Template

```
Base query: {base_query}

Conversation so far:
{hist_txt}

Slot to clarify: {slot}

Ask ONE question to determine the user's requirement for {slot}. Keep it concise.
```

D.5. roleplayer_answer: User Prompt Template

```
Base query the user started with:
{base_query}

Persona (all you know):
{persona}

Assistant asked:
{question}

Answer as the user, using only persona information.
```

D.6. extract_constraint_and_weight: User Prompt Template

```
Slot: {slot}
User answer: {user_answer}

Return JSON:
{
```

```
  "constraint": null OR {"column": "{slot}", "op": "=="|">="|"<="| "==", "value": string|number},
  "weight": number
}

Rules:
- If user has no preference / doesn't care, output constraint:null and weight 0.0.
- If slot is numeric (Year, city mpg, highway MPG, MSRP):
  - Use >= for minimum requirements, <= for maximum budget, == only if explicitly exact.
  - value MUST be an integer.
- If slot is categorical (Engine Fuel Type, Vehicle Size, Model):
  - Use op == and value as string.
- weight in [0,1]: must-have ~0.9-1.0; strongly prefer ~0.7-0.9; nice-to-
have ~0.4-0.7; flexible/compromise ~0.0-0.3.
Output ONLY JSON.
```

D.7. System Prompt Suffixes (Strict Output Enforcement)

D.7.1. STRICT_SUFFIX_1

```
IMPORTANT: Output MUST start with '{' and end with '}' and contain NOTHING else.
```

D.7.2. STRICT_SUFFIX_2

```
FINAL WARNING: ONLY output JSON. No markdown. No code fences. No commentary.
```

E. Evaluation

For evaluation details refer to Table 6

F. Example Dialogue

- **User:** I am looking for a Chevrolet Extended Cab Pickup with a AUTOMATIC transmission and rear wheel drive.
- **Assistant:** Do you have a specific engine fuel type preference for the Chevrolet Extended Cab Pickup?
- **User:** I'd like it to run on flex-fuel (unleaded/E85), but I'm flexible there if everything else fits. That's a nice-to-have.
- **Assistant:** Do you have a specific MSRP range in mind for the Chevrolet Extended Cab Pickup?
- **User:** Must-have. I'm pretty firm on keeping MSRP at no more than $24,515, because budget discipline matters most to me.
- **Assistant:** Do you have a specific vehicle size preference for the Chevrolet Extended Cab Pickup?
- **User:** I strongly prefer it to be a Large truck, though I could compromise a little on size if the overall deal is right.
- **Assistant:** What is your desired highway MPG for the Chevrolet Extended Cab Pickup, and how important is this to you?
- **User:** I'd like it to have at least 20 highway MPG, but that's more of a "nice floor" than a hard line.

Algorithm 1: MUS-2/4 (any-repair)

```
PROCEDURE GenerateInstances(target):
    results ← ∅
    used ← ∅
    while |results| < target do
        U ← SelectUnusedBaseSlice(used) //|U| within bounds
        if U = null then
            break
        P ← BuildCandidatePredicates(U)
        (a, b) ← FindDropAnyOnePair(P) //(a ∧ b) = ∅,  a ≠ ∅,  b ≠ ∅, prefer distinct attributes
        if (a, b) = null then
            continue
        used ← used ∪ {SignatureOf(U)}
        (w_a, w_b) ← SampleNormalizedWeights()
        relaxed ← arg min{(a, w_a), (b, w_b)} //by weight
        kept ← ({a, b} \ {relaxed.predicate})
        C ← { x ∈ U : x satisfies all predicates in kept }
        if C = ∅ then
            results ← results ∪ {Instance(U, {a, b}, {w_a, w_b}, relaxed, null)}
            continue
        foreach x ∈ C do
            score[x] ← 0
            foreach (p, w) ∈ {(a, w_a), (b, w_b)} do
                score[x] ← score[x] + w · SoftSatisfaction(x, p, U)
        best ← arg max_{x∈C} (score[x]) //tie_breaker = lowest_cost
        results ← results ∪ {Instance(U, {a, b}, {w_a, w_b}, relaxed, best)}
    RETURN results

PROCEDURE BuildCandidatePredicates(U):
    P ← ∅
    foreach chosen categorical attribute do
        foreach value with sufficient support in U do
            p ← EqualityPredicate(value)
            if p covers all of U or none of U then
                continue
            P ← P ∪ {p}
    foreach chosen numeric attribute with direction (≥ or ≤) do
        T ← QuantileThresholds(U) //with caps/filters
        foreach threshold in T do
            p ← InequalityPredicate(threshold)
            if p covers all of U or none of U then
                continue
            P ← P ∪ {p}
    P ← Deduplicate(P)
    P ← FilterBySupport(P)
    P ← CapPerAttributeAndTotal(P)
    RETURN P

PROCEDURE FindDropAnyOnePair(P):
    //Try many random pairs from P:
    foreach many random pairs (a, b) drawn from P do
        if attributes distinct and (a ∧ b) = ∅ and a ≠ ∅ and b ≠ ∅ then
            RETURN (a, b)
    //Fallback exhaustive search with same condition:
    foreach all pairs (a, b) in P × P do
        if attributes distinct and (a ∧ b) = ∅ and a ≠ ∅ and b ≠ ∅ then
            RETURN (a, b)
    RETURN null

FUNCTION SoftSatisfaction(x, p, U):
    if p is equality then
        RETURN 1[x matches p]
    else if p is "at least" then
        RETURN NormalizedValue(x.attribute over U)
    else if p is "at most" then
        RETURN 1 − NormalizedValue(x.attribute over U)
```

Algorithm 2: MUS-4 (one unique repair)

```
PROCEDURE GenerateOneFixInstances(target):
    results ← ∅
    used ← ∅
    while |results| < target do
        U ← SelectUnusedBaseSlice(used) //|U| within bounds
        if U = null then
            break
        P ← BuildCandidatePredicates(U)
        (Q, r) ← FindOneFixQuad(P) //Q has 4 predicates, r is unique repair index
        if (Q, r) = null then
            continue
        used ← used ∪ {SignatureOf(U)}
        W ← AssignWeightsWithUniqueLowest(Q, r) //W[r] strictly smallest; normalize
        keep ← Q \ {Q[r]}
        C ← { x ∈ U : x satisfies all predicates in keep }
        if C = ∅ then
            continue
            //should not occur if constructed correctly
        foreach x ∈ C do
            score[x] ← 0
            foreach (p, w) ∈ Zip(Q, W) do
                score[x] ← score[x] + w · SoftSatisfaction(x, p, U)
        best ← arg max_{x∈C} (score[x]) //tie_breaker = lowest_cost
        results ← results ∪ {Instance(U, Q, W, repair = Q[r], recommendation = best)}
    RETURN results

PROCEDURE FindOneFixQuad(P):
    T ← ∅
    foreach triple (i, j, k) from P do
        if RequireDistinctAttributes and not all distinct(i, j, k) then
            continue
        bits ← IntersectBits(i, j, k)
        if bits ≠ 0 then
            T ← T ∪ {(i, j, k, bits)} //triple feasible
    if T = ∅ then
        RETURN null
    Shuffle(T)
    foreach (i, j, k, bits_ijk) ∈ T do
        foreach l in indices(P) excluding {i, j, k} do
            if RequireDistinctAttributes and not all distinct(i, j, k, l) then
                continue
            if (bits_ijk ∩ Bits(l)) ≠ 0 then
                continue
                //full 4-way must be empty
            if IntersectBits(i, j, l) ≠ 0 then
                continue
            if IntersectBits(i, k, l) ≠ 0 then
                continue
            if IntersectBits(j, k, l) ≠ 0 then
                continue
            Q ← [Predicate(i), Predicate(j), Predicate(k), Predicate(l)]
            r ← index of l within Q //dropping Q[r] yields feasible triple
            RETURN (Q, r)
    RETURN null

PROCEDURE AssignWeightsWithUniqueLowest(Q, r):
    W ← array of 4 weights
    W[r] ← SampleSmall()
    foreach index t ≠ r do
        W[t] ← SampleLarge()
    Normalize(W) //sum to 1
    RETURN W

FUNCTION SoftSatisfaction(x, p, U):
    if p is equality then
        RETURN 1[x matches p]
    else if p is "at least" then
        RETURN NormalizedValue(x.attribute over U)
    else if p is "at most" then
        RETURN 1 − NormalizedValue(x.attribute over U)
```

Table 6. Evaluation metrics used in our experiments

Category	Metric	What it measures	How computed
Dialogue completion	Avg #Slots Provided	How many constraint slots the agent was asked to fill	`mean(len(slots_provided_to_agent))`
Dialogue completion	Avg #Constraints Parsed	How many constraints the system actually extracted	`mean(len(parsed_additional_constraints))`
Dialogue completion	Slot Completion Rate	Fraction of provided slots that resulted in a parsed constraint	`total_parsed/total_slots`
Dialogue completion	Per-slot Completion Rate	Which slots are frequently missed	`foreachslot:parsed_count/asked_count`
Planning outcome	SAT (No Relaxation) Rate	Full-query feasibility success	`count(status=="SAT_no_relaxation")/N`
Planning outcome	SAT (After Relaxation) Rate	Success after relaxing $\geq$ 1 constraint	`count(status=="SAT_after_relaxation")/N`
Planning outcome	UNSAT Rate	Failed even after trying relaxation	`count(status=="UNSAT_even_after_relaxation")/N`
Planning outcome	Recommendation Produced Rate	Whether the system produced any car	`count(recommended_car!=null)/N`
Oracle agreement	Relax Match Rate	Whether relaxed constraint matches oracle relaxed constraint	`match(relaxed_constraint, oracle_relaxed) overcomparableinstances`
Oracle agreement	Top-1 Car Match | Relax-Correct	Car match, only when relaxation matches oracle	`match(car_key(agent),car_key(oracle))onrelax-correctsubset`